Sommative module 03

Explication du code, question par question — pour préparer l'entretien.

Comment utiliser ce document

Chaque question a quatre parties : le code avec les numéros de ligne, ce que fait chaque ligne, pourquoi le code est écrit de cette façon, et les questions que le prof risque de poser avec une réponse courte. Le meilleur exercice : ouvrir le fichier dans PyCharm, cacher ce document, et expliquer chaque ligne à voix haute. Ensuite, vérifier ici.

Les fichiers se trouvent dans `sommative/module_03/` : `q1.py` à `q6.py`.

Les notions utilisées dans les 6 programmes

Variable	Un nom qui garde une valeur. `jours = 365` met 365 dans la boîte `jours`. Le = veut dire « mettre dans », pas « est égal à ».
int	Nombre entier : `365`, `0`, `4`. Utilisé pour compter (tirages, parties, heures).
float	Nombre à virgule : `4.00`, `273.15`, `5.2`. Utilisé pour l'argent et les températures.
str	Du texte, entre guillemets : `"oui"`, `"pierre"`.
bool	`True` ou `False`. C'est le résultat d'une comparaison comme `montant < 50`.
list	Plusieurs valeurs entre crochets : `["pierre", "papier", "ciseaux"]` (question 5).
input()	Affiche une question et attend que l'utilisateur tape. Renvoie **toujours** du texte (str), même si on tape un chiffre.
int() / float()	Transforment le texte en nombre. Sans ça, on ne peut pas faire de calcul avec ce que l'utilisateur a tapé.
print() et f"..."	Le `f` devant les guillemets permet de mettre une variable entre `{ }`. `{prix:.2f}` affiche le nombre avec 2 décimales (4 → 4.00). `:.1f` = 1 décimale.
Opérateurs	`+ - * /` pour calculer. `/` donne toujours un float. `< <= > >= == !=` pour comparer. == compare, = range une valeur.
and / or / not	and : les deux doivent être vrais. or : au moins un. `not in` : vrai si la valeur n'est pas dans la liste.
+=	`compte1 += 1` est la version courte de `compte1 = compte1 + 1`.
if / elif / else	Python teste les conditions dans l'ordre et exécute **seulement le premier bloc vrai**. `else` attrape tout le reste.
Indentation	Les 4 espaces au début d'une ligne disent à Python que la ligne fait partie du bloc au-dessus (dans le `if`, dans la boucle…).
for i in range(n)	Répète le bloc n fois. `i` prend les valeurs 0, 1, 2, … n-1.
while True	Répète sans fin. `break` sort de la boucle. `continue` saute le reste du tour et recommence au début de la boucle.
import	Charge un module. `random.randint(1, 4)` : nombre au hasard de 1 à 4 (1 et 4 inclus). `random.choice(liste)` : un élément au hasard. `math.ceil(2.3)` : arrondit vers le haut → 3.
.lower()	Met un texte en minuscules : `"Oui"` devient `"oui"`.
#	Commentaire. Python ne le lit pas ; il sert à expliquer le code à la personne qui le lit.

Une limite à connaître (valable pour tous les programmes)

Si l'utilisateur tape des lettres là où on attend un nombre (par exemple `abc` pour la température), `float()` ou `int()` ne peut pas convertir et le programme s'arrête avec une erreur `ValueError`. Je ne l'ai pas géré parce que la consigne ne le demande pas et que l'outil pour ça (`try` / `except`) n'a pas été vu dans ce module. Si le prof en parle, c'est la bonne réponse.

Question 1 — Heures, minutes et secondes dans une année

But : partir de 365 jours et calculer les heures, les minutes et les secondes.

```
1  # Question 1 : heures, minutes et secondes dans une année
2
3  jours = 365
4
5  heures = jours * 24       # 1 jour = 24 heures
6  minutes = heures * 60     # 1 heure = 60 minutes
7  secondes = minutes * 60   # 1 minute = 60 secondes
8
9  print("Dans une année de 365 jours :")
10 print(f"Jours : {jours}")
11 print(f"Heures : {heures}")
12 print(f"Minutes : {minutes}")
13 print(f"Secondes : {secondes}")
```

Ligne par ligne

Ligne	Ce qu'elle fait
1	Commentaire qui dit de quoi parle le fichier. Python l'ignore.
3	`jours = 365` : la seule valeur donnée. C'est un int. Tout le reste est calculé à partir d'elle.
5	`heures = jours * 24` : 365 × 24 = 8760.
6	`minutes = heures * 60` : on part du résultat d'avant, 8760 × 60 = 525 600.
7	`secondes = minutes * 60` : 525 600 × 60 = 31 536 000.
9	Affiche un titre. C'est du texte fixe, pas besoin de f-string.
10 à 13	Une f-string par valeur. `{jours}` est remplacé par la valeur de la variable.

Pourquoi je l'ai écrit comme ça

La consigne interdit d'écrire les réponses directement. Chaque valeur est donc calculée à partir de la précédente : jours → heures → minutes → secondes, dans l'ordre demandé.

Tout est en int : on multiplie des entiers, donc le résultat est un entier. Pas besoin de décimales.

Questions que le prof pourrait poser

Pourquoi ne pas écrire directement secondes = 31536000 ?

La consigne l'interdit. En plus, si je change `jours` à 366 (année bissextile), toutes les autres valeurs se recalculent toutes seules.

Quel est le type de la variable heures ?

int. Un int multiplié par un int donne un int.

Pourquoi minutes = heures * 60 et pas jours * 24 * 60 ?

Les deux donnent le même résultat. J'ai suivi les étapes de la consigne : chaque ligne fait une seule conversion, c'est plus facile à lire et à vérifier.

Exemple d'exécution

```
Dans une année de 365 jours :
Jours : 365
Heures : 8760
Minutes : 525600
Secondes : 31536000
```

Question 2 — Conversion de température

But : demander une température en Celsius et l'afficher en Fahrenheit et en Kelvin.

```
1  # Question 2 : conversion de température
2
3  # float() accepte les nombres entiers et les nombres à virgule
4  celsius = float(input("Entrez une température en Celsius : "))
5
6  # -273.15 °C est la température la plus froide possible
7  if celsius < -273.15:
8      print("Erreur : la température ne peut pas être plus basse que -273.15 °C.")
9  else:
10     fahrenheit = (celsius * 9 / 5) + 32
11     kelvin = celsius + 273.15
12
13     print(f"{celsius:.2f} °C = {fahrenheit:.2f} °F = {kelvin:.2f} K")
```

Ligne par ligne

Ligne	Ce qu'elle fait
4	`input()` pose la question et renvoie du texte. `float()` le transforme en nombre à virgule. Le résultat va dans `celsius`.
7	`if celsius < -273.15:` vérifie si la température est sous le zéro absolu (la plus froide qui existe).
8	Si oui, on affiche une erreur. Rien d'autre ne se calcule.
9	`else:` : tout ce qui est en dessous ne s'exécute que si la température est valide.
10	Formule de la consigne : F = (C × 9 / 5) + 32.
11	Formule de la consigne : K = C + 273.15.
13	Affiche les trois températures. `:.2f` donne 2 décimales, comme demandé.

Pourquoi je l'ai écrit comme ça

J'utilise `float()` et pas `int()` parce que la consigne dit d'accepter les nombres décimaux. Avec `int()`, taper `25.5` ferait planter le programme.

Les calculs sont dans le `else` : ça ne sert à rien de convertir une température qui n'existe pas.

`:.2f` change seulement l'affichage. La valeur dans la variable n'est pas modifiée.

Questions que le prof pourrait poser

Pourquoi < et pas <= ?

La consigne dit de refuser une température « inférieure à » -273.15. -273.15 elle-même est donc acceptée.

Les parenthèses dans (celsius * 9 / 5) + 32 sont-elles obligatoires ?

Non. Python fait déjà * et / avant +. Je les ai mises pour que la formule ressemble à celle de la consigne et soit plus claire.

Qu'est-ce qui se passe si on tape 25 (sans virgule) ?

`float("25")` donne 25.0. Ça marche avec les entiers et les décimaux.

Exemple d'exécution

```
Entrez une température en Celsius : 25
25.00 °C = 77.00 °F = 298.15 K

Entrez une température en Celsius : -300
Erreur : la température ne peut pas être plus basse que -273.15 °C.
```

Question 3 — Facture avec rabais et taxes

But : appliquer un rabais selon le montant, puis la TVH de 13 % sur le montant réduit.

```
1  # Question 3 : facture avec rabais et taxes
2
3  montant = float(input("Montant de l'achat avant taxes : "))
4
5  # On choisit le rabais selon le montant
6  if montant < 50:
7      rabais = 0
8  elif montant <= 100:
9      rabais = 5
10 else:
11     rabais = 10
12
13 economie = montant * rabais / 100
14 sous_total = montant - economie
15 taxe = sous_total * 0.13          # TVH de 13 % sur le montant après rabais
16 total = sous_total + taxe
17
18 print(f"Montant initial : {montant:.2f} $")
19 print(f"Rabais : {rabais} %")
20 print(f"Montant économisé : {economie:.2f} $")
21 print(f"Sous-total après rabais : {sous_total:.2f} $")
22 print(f"TVH (13 %) : {taxe:.2f} $")
23 print(f"Total final : {total:.2f} $")
```

Ligne par ligne

Ligne	Ce qu'elle fait
3	Lit le montant et le transforme en float (un prix peut avoir des cents).
6 à 7	Moins de 50 $: rabais de 0 %.
8 à 9	Sinon, si le montant est 100 $ ou moins : 5 %. On arrive ici seulement si le premier test est faux, donc on sait déjà que le montant est au moins 50 $.
10 à 11	Tout le reste (plus de 100 $) : 10 %.
13	Montant économisé = montant × rabais / 100. Exemple : 120 × 10 / 100 = 12.
14	Sous-total = montant − économie.
15	La taxe est calculée sur le **sous-total**, pas sur le montant de départ. C'est ce que dit la consigne.
16	Total = sous-total + taxe.
18 à 23	Affiche les 6 informations demandées. Les montants ont 2 décimales (`:.2f`). Le rabais est un entier, il s'affiche sans décimales.

Pourquoi je l'ai écrit comme ça

J'ai gardé le rabais en pourcentage (0, 5 ou 10) plutôt qu'en décimal (0.05). Comme ça, je peux l'afficher directement : « Rabais : 10 % ».

J'utilise `if / elif / else` parce qu'un seul rabais doit s'appliquer. Python s'arrête à la première condition vraie.

Questions que le prof pourrait poser

Quel rabais pour exactement 50 $? Et 100 $?

5 % dans les deux cas. 50 < 50 est faux, donc on passe au `elif` ; 50 <= 100 est vrai. Pour 100 $, <= inclut 100. La consigne dit « de 50 $ à 100 $ inclusivement ».

Pourquoi elif et pas trois if séparés ?

Avec des `if` séparés, Python testerait toutes les conditions et pourrait changer le rabais plusieurs fois. Avec `elif`, il prend seulement la première qui est vraie.

Pourquoi pas elif 50 <= montant <= 100 ?

Ça marcherait aussi. Ce n'est pas nécessaire : si on arrive au `elif`, le premier `if` a déjà éliminé les montants sous 50 $.

Pourquoi la taxe après le rabais ?

La consigne le demande, et c'est comme ça en vrai : on paie la taxe sur ce qu'on paie réellement.

Exemple d'exécution

```
Montant de l'achat avant taxes : 120
Montant initial : 120.00 $
Rabais : 10 %
Montant économisé : 12.00 $
Sous-total après rabais : 108.00 $
TVH (13 %) : 14.04 $
Total final : 122.04 $
```

Question 4 — Simulation de tirages aléatoires

But : tirer beaucoup de nombres de 1 à 4, compter chaque résultat et calculer les pourcentages.

```python
# Question 4 : simulation de tirages aléatoires entre 1 et 4

import random

nombre_tirages = int(input("Combien de tirages voulez-vous faire ? "))

# Un compteur pour chaque nombre
compte1 = 0
compte2 = 0
compte3 = 0
compte4 = 0

for i in range(nombre_tirages):
    tirage = random.randint(1, 4)
    if tirage == 1:
        compte1 += 1
    elif tirage == 2:
        compte2 += 1
    elif tirage == 3:
        compte3 += 1
    else:
        compte4 += 1

if nombre_tirages > 0:
    print(f"1 : {compte1} fois — {compte1 / nombre_tirages * 100:.1f} %")
    print(f"2 : {compte2} fois — {compte2 / nombre_tirages * 100:.1f} %")
    print(f"3 : {compte3} fois — {compte3 / nombre_tirages * 100:.1f} %")
    print(f"4 : {compte4} fois — {compte4 / nombre_tirages * 100:.1f} %")

    # On cherche le nombre tiré le plus souvent
    plus_souvent = 1
    plus_grand = compte1
    if compte2 > plus_grand:
        plus_souvent = 2
        plus_grand = compte2
    if compte3 > plus_grand:
        plus_souvent = 3
        plus_grand = compte3
    if compte4 > plus_grand:
        plus_souvent = 4
        plus_grand = compte4

    print(f"Le nombre tiré le plus souvent : {plus_souvent}")
else:
    print("Aucun tirage effectué.")
```

Ligne par ligne

Ligne	Ce qu'elle fait
3	`import random` : charge le module qui fait le hasard.
5	Lit le nombre de tirages. `int()` parce qu'on ne peut pas faire 2.5 tirages, et `range()` a besoin d'un entier.
8 à 11	Un compteur par nombre, tous à 0 avant la boucle. Sans ça, `compte1 += 1` ferait une erreur : la variable n'existerait pas encore.
13	`for i in range(nombre_tirages):` répète le bloc autant de fois que demandé. `i` n'est pas utilisé, il sert seulement à compter les tours.
14	`random.randint(1, 4)` donne 1, 2, 3 ou 4 (1 et 4 inclus).
15 à 22	Ajoute 1 au bon compteur. Le `else` correspond à 4, parce que c'est la seule valeur qui reste.
24	On vérifie qu'il y a eu au moins un tirage. Sinon, la ligne 25 diviserait par 0 et le programme planterait.
25 à 28	Pourcentage = compteur / total × 100. `:.1f` donne 1 décimale, comme l'exemple de la consigne (24.3 %).
31 à 32	On commence en supposant que 1 est le plus fréquent. `plus_grand` garde le plus grand compteur trouvé jusqu'ici.
33 à 41	On compare chaque autre compteur au plus grand. S'il est plus grand, il le remplace.
43	Affiche le nombre tiré le plus souvent.
44 à 45	Si on a demandé 0 tirage, on affiche un message au lieu de calculer.

Pourquoi je l'ai écrit comme ça

Quatre variables séparées, c'est simple à lire quand il n'y a que 4 valeurs. Une liste serait plus courte, mais pas nécessaire ici.

Pour trouver le plus fréquent, j'ai utilisé la méthode « le plus grand jusqu'ici » : on garde un gagnant et on le remplace dès qu'on trouve mieux.

Questions que le prof pourrait poser

Pourquoi des if aux lignes 33, 36 et 39 et pas des elif ?

Il faut tester les trois. Avec `elif`, Python s'arrêterait à la première condition vraie et pourrait rater un compteur encore plus grand.

Et s'il y a une égalité entre deux nombres ?

J'utilise `>` et pas `>=`, donc on garde le premier trouvé (le plus petit nombre).

Pourquoi les résultats changent à chaque exécution ?

C'est du hasard. Plus on fait de tirages, plus chaque pourcentage se rapproche de 25 %.

Qu'est-ce qui arrive avec 0 tirage ?

La boucle ne tourne pas, et la ligne 24 évite la division par zéro : on affiche « Aucun tirage effectué. »

Exemple d'exécution

```
Combien de tirages voulez-vous faire ? 1000
1 : 249 fois — 24.9 %
2 : 252 fois — 25.2 %
3 : 262 fois — 26.2 %
4 : 237 fois — 23.7 %
Le nombre tiré le plus souvent : 3
```

Question 5 — Pierre, papier, ciseaux

But : jouer contre l'ordinateur jusqu'à ce que le joueur tape « quitter », et compter les victoires, défaites et égalités.

```
# Question 5 : jeu Pierre, papier, ciseaux

import random

choix_possibles = ["pierre", "papier", "ciseaux"]

victoires = 0
defaites = 0
egalites = 0

while True:
    joueur = input("Pierre, papier ou ciseaux ? (quitter pour arrêter) : ").lower()

    if joueur == "quitter":
        break

    # Un choix invalide ne compte pas comme une partie
    if joueur not in choix_possibles:
        print("Choix invalide, essayez encore.")
        continue

    ordinateur = random.choice(choix_possibles)
    print(f"Vous : {joueur} | Ordinateur : {ordinateur}")

    if joueur == ordinateur:
        print("Égalité !")
        egalites += 1
    # Pierre bat ciseaux, ciseaux battent papier, papier bat pierre
    elif (joueur == "pierre" and ordinateur == "ciseaux"
          or joueur == "ciseaux" and ordinateur == "papier"
          or joueur == "papier" and ordinateur == "pierre"):
        print("Vous gagnez !")
        victoires += 1
    else:
        print("Vous perdez.")
        defaites += 1

print()
print(f"Victoires : {victoires}")
print(f"Défaites : {defaites}")
print(f"Égalités : {egalites}")
print(f"Total de parties : {victoires + defaites + egalites}")
```

Ligne par ligne

Ligne	Ce qu'elle fait
3	`import random` pour le choix de l'ordinateur.
5	La liste des 3 choix valides. Elle sert deux fois : pour vérifier le choix du joueur et pour le choix de l'ordinateur.
7 à 9	Les trois compteurs, à 0 au départ.
11	`while True:` répète sans fin. La seule sortie est le `break` de la ligne 15.
12	Lit le choix du joueur. `.lower()` met en minuscules : « Pierre » et « PIERRE » deviennent « pierre ».
14 à 15	Si le joueur tape « quitter », `break` sort de la boucle. Le programme continue à la ligne 38.
18	`not in` est vrai si le choix n'est pas dans la liste (par exemple « banane »).
19 à 20	Message d'erreur, puis `continue` : on retourne au début de la boucle **sans** toucher aux compteurs. Un choix invalide ne compte donc pas comme une partie.
22	`random.choice()` prend un élément au hasard dans la liste.
23	Affiche les deux choix, comme demandé.
25 à 27	Même choix = égalité.
29 à 31	Les trois façons de gagner, reliées par `or`. Il suffit qu'une soit vraie. Les parenthèses autour permettent d'écrire la condition sur plusieurs lignes.
34 à 36	Si ce n'est pas une égalité et que le joueur n'a pas gagné, il a perdu.
38	`print()` vide : saute une ligne pour que le résumé soit lisible.
39 à 42	Le résumé. Le total est l'addition des trois compteurs, pas besoin d'un quatrième compteur.

Pourquoi je l'ai écrit comme ça

J'utilise `while` et pas `for` parce qu'on ne sait pas d'avance combien de parties le joueur va faire.

Pour la défaite, je n'écris pas les trois façons de perdre : si ce n'est ni une égalité ni une victoire, c'est forcément une défaite. Le `else` suffit.

Questions que le prof pourrait poser

Quelle est la différence entre break et continue ?

`break` sort complètement de la boucle (fin du jeu). `continue` saute seulement le reste du tour actuel et recommence au début de la boucle.

Pourquoi le choix invalide n'est pas compté ?

Le `continue` de la ligne 20 renvoie au début de la boucle avant les lignes qui ajoutent 1 aux compteurs.

Que se passe-t-il si le joueur tape « Quitter » avec une majuscule ?

`.lower()` le transforme en « quitter », donc le jeu s'arrête quand même.

Dans la condition de victoire, comment Python lit and et or ?

`and` est évalué avant `or`. Chaque ligne est donc un couple (choix du joueur ET choix de l'ordinateur), et les trois couples sont reliés par `or`.

Exemple d'exécution

```
Pierre, papier ou ciseaux ? (quitter pour arrêter) : pierre
Vous : pierre | Ordinateur : ciseaux
Vous gagnez !
Pierre, papier ou ciseaux ? (quitter pour arrêter) : banane
Choix invalide, essayez encore.
Pierre, papier ou ciseaux ? (quitter pour arrêter) : quitter

Victoires : 1
Défaites : 0
Égalités : 0
Total de parties : 1
```

Question 6 — Frais de stationnement

But : calculer le prix selon la durée, avec un maximum de 20 $ et un rabais de 20 % pour les voitures électriques.

```
# Question 6 : frais de stationnement

import math

electrique = input("Le véhicule est-il électrique ? (oui/non) : ").lower()
duree = float(input("Nombre d'heures de stationnement : "))

# math.ceil arrondit vers le haut : 2.3 h devient 3 h
heures_facturees = math.ceil(duree)

# Il faut payer au moins 1 heure
if heures_facturees < 1:
    heures_facturees = 1

# Première heure : 4.00 $, chaque heure en plus : 2.50 $
prix = 4.00 + (heures_facturees - 1) * 2.50

# Le prix ne peut pas dépasser 20.00 $
if prix > 20.00:
    prix = 20.00

# Rabais de 20 % pour un véhicule électrique
if electrique == "oui":
    rabais = prix * 0.20
else:
    rabais = 0.00

total = prix - rabais

print(f"Durée : {duree} heures")
print(f"Heures facturées : {heures_facturees}")
print(f"Prix avant rabais : {prix:.2f} $")
print(f"Rabais véhicule électrique : {rabais:.2f} $")
print(f"Total : {total:.2f} $")
```

Ligne par ligne

Ligne	Ce qu'elle fait
3	`import math` pour utiliser `math.ceil()`.
5	Demande si la voiture est électrique. `.lower()` pour accepter « Oui » ou « OUI ».
6	Lit la durée en float, parce qu'elle peut avoir des décimales (5.2 h).
9	`math.ceil()` arrondit **vers le haut** : 5.2 → 6, 2.3 → 3. Un nombre déjà entier ne change pas : 4.0 → 4. C'est la règle « toute heure commencée compte ».
12 à 13	Si on tape 0, il faut quand même payer 1 heure. Sans ces lignes, la formule de la ligne 16 donnerait 1.50 $, ce qui n'a pas de sens.
16	4.00 $ pour la première heure, plus 2.50 $ pour chaque heure **après** la première. C'est pour ça qu'on fait `heures_facturees - 1`. Pour 6 h : 4 + 5 × 2.50 = 16.50.
19 à 20	Si le prix dépasse 20 $, on le ramène à 20 $ (maximum quotidien).
23 à 26	Rabais de 20 % si la réponse est « oui ». Sinon, le rabais vaut 0.
28	Total = prix − rabais.
30 à 34	Affiche les 5 informations demandées, avec 2 décimales pour l'argent.

Pourquoi je l'ai écrit comme ça

J'ai choisi `math.ceil()` parce que `round()` et `int()` ne marchent pas ici : `round(2.3)` donne 2 et `int(2.3)` donne aussi 2. On veut 3.

Le maximum de 20 $ est appliqué **avant** le rabais. Le maximum est le prix du stationnement ; le rabais électrique s'applique ensuite sur ce prix. Une voiture électrique paie donc au plus 16.00 $.

Toute réponse autre que « oui » est traitée comme « non ». C'est le plus simple, et la consigne ne demande pas de vérifier la réponse.

Questions que le prof pourrait poser

Pourquoi heures_facturees - 1 ?

La première heure est déjà comptée dans le 4.00 $. Les 2.50 $ s'appliquent seulement aux heures suivantes.

À partir de combien d'heures le maximum de 20 $ s'applique ?

7 h coûtent 4 + 6 × 2.50 = 19.00 $. 8 h coûteraient 21.50 $, donc on paie 20.00 $ à partir de 8 heures.

Pourquoi ne pas utiliser round() ?

`round()` arrondit au plus proche : 2.3 deviendrait 2. La consigne dit qu'une heure commencée compte au complet, donc il faut toujours arrondir vers le haut.

Pourquoi les lignes 12 et 13 ?

Pour le cas où on tape 0 (ou un nombre négatif). Sans elles, le prix serait 1.50 $ ou moins. Avec elles, on paie au moins la première heure.

Exemple d'exécution

```
Le véhicule est-il électrique ? (oui/non) : oui
Nombre d'heures de stationnement : 5.2
Durée : 5.2 heures
Heures facturées : 6
Prix avant rabais : 16.50 $
Rabais véhicule électrique : 3.30 $
Total : 13.20 $
```